

Response to Agentc Feedback

Thank you for the thorough and technically rigorous review. The feedback is well-structured and identifies the right pressure points. This response addresses each major concern directly and updates the project framing where warranted.

On the DAG Planning Paradox

This is the strongest critique and the one most worth taking seriously. The feedback is correct: for open-ended tasks, the execution DAG cannot be fully specified upfront. Agentc workflows are fundamentally reactive — tool call results reshape the plan, and the structure of the problem is often only revealed through execution.

However, we think the paradox is a spectrum rather than a binary. At one end: fully open-ended tasks (debug an unknown crash) where the DAG is almost entirely discovered during execution. At the other: structured pipelines (analyze N files for security vulnerabilities) where the DAG is embarrassingly parallel and fully known upfront. The compiler analogy holds fully for the latter and partially for the former.

The architectural implication is to treat Agentc as a **runtime optimizer** rather than a static compiler — intercepting and optimizing each call as it is issued rather than pre-compiling the entire plan. This is closer to a JIT compiler than an AOT compiler, which is arguably a stronger analogy: the optimizer operates on a rolling window of the execution graph, applying rewrites to subgraphs that are known rather than waiting for the full plan to be resolved.

Concretely, this means:

- **Static optimization** applies to subgraphs that are fully specified — parallelizing independent tool calls, downgrading models for known-structure nodes, applying `StateDrop` to intermediate results with no downstream consumers.
- **Dynamic optimization** applies at the call boundary — compressing context just before a call fires, routing to the cheapest model that meets a quality threshold, memoizing semantically equivalent calls.

The research narrative becomes: *prove the optimizer on fixed-structure pipelines first, then characterize the degradation as structure becomes less known*. That is a cleaner empirical story than attempting to solve the fully reactive case from the outset.

On Reactive Execution

Related to the above. The feedback notes that static DAGs optimized before execution cannot handle surprises without re-planning, and that constant re-planning is just an agent. This is true.

The reframe: Agentc does not try to eliminate re-planning. It tries to make each individual planning and execution step cheaper. Even in a fully reactive workflow, every LLM call still has a context window that can be compressed, a model tier that can be selected, a cache that can be hit. The optimizer does not need to see the whole plan to apply these optimizations — it only needs to see the next call. The DAG IR is still useful as a local structure even when the global structure is unknown.

This pushes the project toward an **intercept-and-optimize** architecture: Agentc sits as a proxy layer that every LLM call passes through, applying whatever optimizations are locally applicable without requiring global plan visibility.

On the Observability Problem

The feedback's data availability table is accurate and useful. The implication — that a custom harness is necessary to get the observability needed for cost model training — aligns with our existing position. This is an argument for building the profiler as a first-class artifact rather than a supporting tool.

A standalone profiler that instruments any agent pipeline and produces a structured execution trace (token counts per call, latency, context utilization, model tier) has immediate utility independent of the optimizer. We agree with the feedback's suggestion that **this ships first**. The profiling dataset itself is an empirical contribution, and it builds the foundation the cost model requires.

Token-level attribution — what the model actually attended to — is not exposed by any provider API. This is a real limitation. The cost model therefore has to work with input token counts and empirical accuracy measurements rather than attention weights. This narrows the model's precision but does not invalidate it: input token count is a strong proxy for cost, and node-level accuracy can be measured through task-specific evaluation rubrics.

On the Abstraction Level Question

The tension between concrete nodes (precise cost estimates, brittle) and abstract nodes

(stable, unpredictable costs) is real. Our current position: nodes should be **typed but not fully specified**. A node type like `CodeAnalysis(file_count: usize, context_budget: TokenCount)` carries enough structure for the cost model to make useful predictions without over-specifying the exact prompt. The cost model learns a distribution over token costs conditioned on node type and parameter values — not a point estimate.

This is analogous to how query optimizers handle uncertain cardinality estimates: they do not need perfect predictions, they need predictions good enough to discriminate between execution strategies. The same bar applies here.

On Competition with RL-Tuned Models

The feedback raises an important distinction: which optimizer rules operate at the runtime level (complementary to model-internal optimization) and which operate at the decision level (potentially competing with it)?

Our current read:

Rule	Level	Relationship to RL
<code>ContextCompress</code>	Decision	Complementary — model doesn't control its own context window
<code>ParallelBranch</code>	Runtime	Complementary — model has no cross-call awareness
<code>ModelDowngrade</code>	Decision	Potentially competing — model routing overlaps with learned behavior
<code>StateDrop</code>	Runtime	Complementary — model cannot prune its own input
<code>DeferredEvaluation</code>	Decision	Complementary — model cannot defer its own calls

`ModelDowngrade` is the rule most likely to conflict with model-internal optimization and warrants the most careful empirical treatment. The others are largely orthogonal to what RL training can influence.

On the Extended Ideas

Several of the proposed extensions are compelling enough to warrant explicit acknowledgment:

Speculative execution / trajectory prediction — profiling real agent traces to predict next tool calls and fire them speculatively on cheaper models is a strong idea with a clear empirical methodology. Agent tool-call sequences are far more predictable than general text, which makes the Markov/n-gram predictor a reasonable starting point. This is a natural Phase 3 extension once the baseline profiling infrastructure exists.

Semantic memoization via LSH — content-addressed caching at the semantic level is independently novel and implementable early. Negative caching (propagating dead-end signals across agents) is particularly interesting and underexplored. This may be worth developing as a standalone contribution if the core optimizer takes longer than expected.

Write-ahead logging for agent state — applying ARIES-style recovery to agent execution is a clean cross-domain insight. The hard part is defining agent state at the right granularity, but the logging infrastructure would be built anyway as part of the profiler.

On Prior Work

We were not previously aware of Kim et al. (2024) at the level of detail described here. Reading it is now the immediate next step. The differentiation the feedback proposes — Kim optimizes call scheduling, Agentc optimizes the execution engine the plan runs on — is the right framing and needs to be internalized before any further documentation or implementation work.

The Anthropic Engineering post on effective harnesses and the Böckeler/Fowler piece on harness engineering are also being pulled now.

Revised Summary of Position

The feedback sharpens rather than invalidates the core thesis. Key updates:

1. **Reframe from static compiler to JIT runtime optimizer.** The DAG IR is a local rather than global structure. The optimizer intercepts calls as they are issued rather than pre-compiling the full plan.
2. **Ship the profiler first.** The profiling dataset is an independent empirical contribution and the foundation everything else requires.
3. **Prove on structured pipelines first.** Fixed-structure tasks are where the compiler analogy holds fully. Characterize degradation as structure decreases — that is the empirical story.

4. **Treat semantic memoization as a parallel track.** Fast to prototype, measurable results, independently publishable.

The research framing the feedback suggests — *Kim et al. optimized tool call scheduling; Agentc optimizes the full inference runtime beneath it* — is clean, defensible, and directly positions this work in the existing literature. We are adopting it.

Thank you again for the depth of engagement. This feedback materially improved the project.